WEJ LAB

The H2O Package

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-14

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Describe what H2O is and why it scales
- Set up H2O in R (Java required)
- Run H2O k-means and AutoML
- Read an AutoML leaderboard

Describing H2O

There is a lot of buzz for machine learning algorithms as well as a requirement for its experts. We all know that there is a significant gap in the skill requirement. The motive of H2O is to provide a platform which made easy for the non-experts to do experiments with machine learning.

Describing H2O

H2O's core code is written in Java that enables the whole framework for multi-threading. Although it is written in Java, it provides interfaces for R, Python and others, thus enabling it to be used efficiently.

In short, we can say that H2O is an open source, in memory, distributed, fast and scalable machine learning and predictive analytics toolkit that facilitates the building and application of machine learning models.

Using H2O

```r
# install.packages("h2o")
library(h2o)
```

```
##
## ----------------------------------------------------------------------
##
## Your next step is to start H2O:
##     > h2o.init()
##
## For H2O package documentation, ask for help:
##     > ??h2o
##
## After starting H2O, you can use the Web UI at http://localhost:54321
## For more information visit https://docs.h2o.ai
##
## ----------------------------------------------------------------------

##
## Attaching package: 'h2o'

## The following objects are masked from 'package:lubridate':
##
##     day, hour, month, week, year

## The following objects are masked from 'package:stats':
##
##     cor, sd, var
```

Setup: H2O needs Java

H2O runs on Java. If `h2o.init()` fails with **"Unable to locate a Java Runtime"**, install a 64-bit JDK (version 17 is a safe choice):

- **Mac (Homebrew):** `brew install --cask temurin@17`
- **Windows / Mac without Homebrew:** download the Temurin 17 installer from adoptium.net
- **Amarel:** `module load java` before starting R

Then **fully restart RStudio** and check that R can see Java:

```
system("java -version")    # should report version 17
```

Still not found? Point R to it (Mac), then restart RStudio again:

```
echo "JAVA_HOME=$(/usr/libexec/java_home -v 17)" >> ~/.Renviron
```

Using H2O

```
h2o.init()
```

```
##  Connection successful!
##
## R is connected to the H2O cluster:
##     H2O cluster uptime:          1 minutes 55 seconds
##     H2O cluster timezone:        America/Denver
##     H2O data parsing timezone:   UTC
##     H2O cluster version:         3.44.0.3
##     H2O cluster version age:     2 years, 9 months and 23 days
##     H2O cluster name:            H2O_started_from_R_evan_buu323
##     H2O cluster total nodes:     1
##     H2O cluster total memory:    11.90 GB
##     H2O cluster total cores:     16
##     H2O cluster allowed cores:   16
##     H2O cluster healthy:         TRUE
##     H2O Connection ip:           localhost
##     H2O Connection port:         54321
##     H2O Connection proxy:        NA
##     H2O Internal Security:       FALSE
```

Using H2O

The h2o.init() command is pretty smart and does a lot of work. At first, it looks for any active H2O instance before starting a new one and then starts a new one when instance are not present.

Using H2O

It does have arguments which helps to accommodate resources to the H2O instance frequently used are:

- **nthreads:** By default, the value of nthreads will be -1 which means the instance can use all the cores of the CPU, we can set the number of cores utilized by passing the value to the argument.
- **max_mem_size:** By passing a value to this argument you can restrict the maximum memory allocated to the instance. Its od string type can pass an argument as '2g' or '2G' for 2 GBs of memory, same when you want to allocate in MBs.

Using H2O

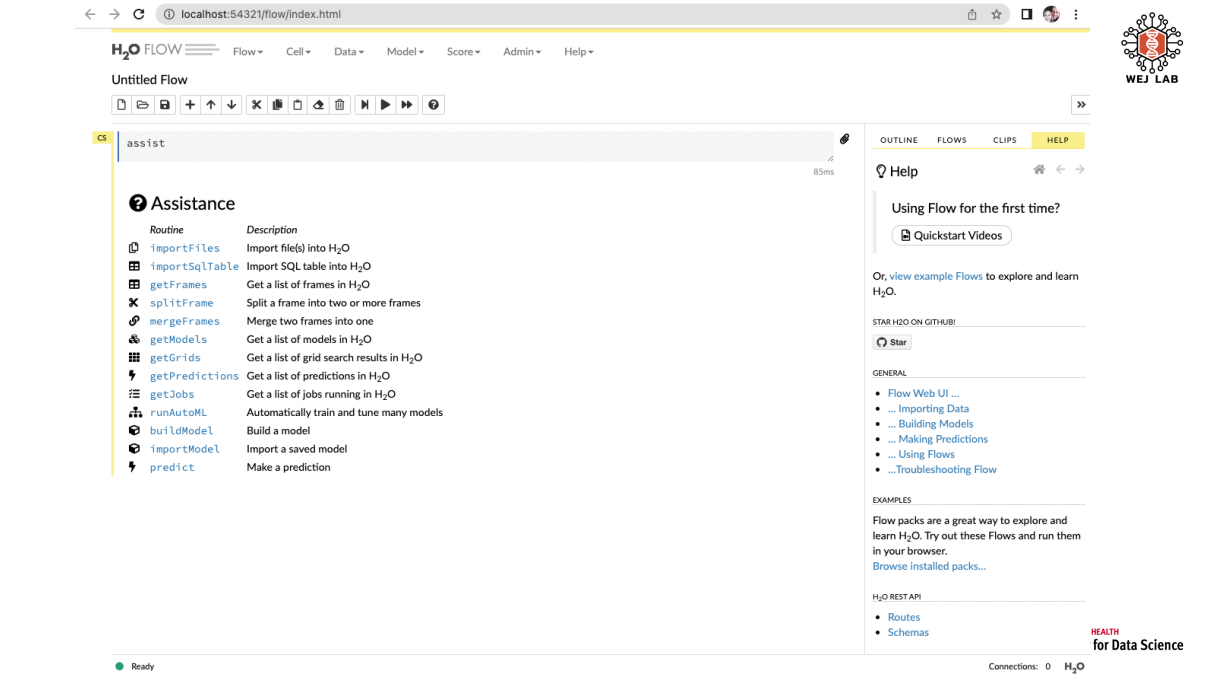
You can access the flow by typing http://localhost:54321 in your browser.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Using H2O

Flow is the name of the web interface that is part of H2O which does not require any extra installations which is written in CoffeeScript (a JavaScript like language). You can use it for doing the following things:

- Upload data directly
- View data uploaded by the client
- Create models directly
- View models created by you or your client
- view predictions
- Run predictions directly



WEJ LAB
HEALTH
for Data Science

H2O AutoML

AutoML helps in automatic training and tuning of many models within a user-specified time limit.

The current version of AutoML function can train and cross-validate a Random Forest, an Extremely-Randomized Forest, a random grid of Gradient Boosting Machines (GBMs), a random grid of Deep Neural Nets, and then trains a Stacked Ensemble using all of the models.

H2O AutoML

When we say AutoML, it should cater to the aspects of data preparation, Model generation, and Ensembles and also provide few parameters as possible so that users can perform tasks with little confusion.

AutoML inputs required arguments **y** and the **training_frame**, with the **x** and **validation frame** as optional arguments. The user can also configure values for **max_runtime_sec** and **max_models**.

H2O AutoML

Additional optional parameters include:

- leaderboard_frame
- nfolds
- fold_columns
- weights_column
- ignored_columns
- stopping_metric
- sort_metric

H2O Kmeans on the iris data

```
iris_h2o <- as.h2o(iris)
```

```
##   |
```

```
iris_h2o['Species'] <- as.factor(iris_h2o['Species'])
predictors <- colnames(iris_h2o)[-length(iris_h2o)]

iris_splits <- h2o.splitFrame(data = iris_h2o,
                              ratios = 0.7, seed = 1234)
train <- iris_splits[[1]]
valid <- iris_splits[[2]]
```

H2O Kmeans on the iris data

```
kmeans_model <- h2o.kmeans(training_frame = train,
                           x = predictors, k = 3,
                           seed = 1)
```

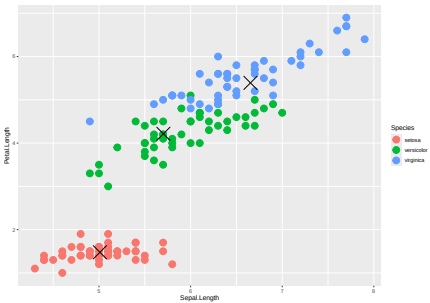
```
##   |
```

```
centers <- h2o.centers(kmeans_model)
centers
```

```
##   sepallength sepalwidth petallength petalwidth
## 1    5.702857   2.637143    4.214286   1.340000
## 2    6.653333   3.051111    5.391111   1.937778
## 3    5.010000   3.436667    1.476667   0.250000
```

H2O Kmeans on the iris data

```
iris %>% ggplot(aes(Sepal.Length,Petal.Length)) +
  geom_point(aes(col=Species), size=5) +
  geom_point(aes(sepallength,petallength),
             col=1, size=10, pch = 4, data=centers)
```



H2O AutoML on the iris data

```
iris_automl <- h2o.automl(x = predictors, y = "Species",
                          training_frame = train,
                          max_runtime_secs = 20, seed = 1,
                          validation_frame = valid)
```

```
##   |                                                                              |
## 09:51:59.883: User specified a validation frame with cross-validation still enabled. Please note that the models will still be vali
## 09:51:59.884: AutoML: XGBoost is not available; skipping it.
## 09:52:00.146: _min_rows param, The dataset size is too small to split for min_rows=100.0: must have at least 200.0 (weighted) rows,
```

```
iris_automl
```

```
## AutoML Details
## ==============
## Project Name: AutoML_3_20261014_95159
## Leader Model ID: GLM_1_AutoML_3_20261014_95159
## Algorithm: glm
##
## Total Number of Models Trained: 67
## Start Time: 2026-10-14 09:52:00 UTC
## End Time: 2026-10-14 09:52:21 UTC
## Duration: 21 s
##
## Leaderboard
## ===========
##                                             model_id mean_per_class_error
## 1                          GLM_1_AutoML_3_20261014_95159           0.02595453
## 2     DeepLearning_grid_1_AutoML_3_20261014_95159_model_1           0.02595453
```

H2O AutoML on the iris data

```
lb <- h2o.get_leaderboard(iris_automl)
head(lb)
```

```
##                                                model_id mean_per_class_error
## 1                          GLM_1_AutoML_3_20261014_95159            0.02595453
## 2    DeepLearning_grid_1_AutoML_3_20261014_95159_model_1            0.02595453
## 3            GBM_grid_1_AutoML_3_20261014_95159_model_23            0.02595453
## 4 StackedEnsemble_BestOfFamily_2_AutoML_3_20261014_95159            0.02595453
## 5             GBM_grid_1_AutoML_3_20261014_95159_model_3            0.02595453
## 6                          DRF_1_AutoML_3_20261014_95159            0.02595453
##      logloss      rmse        mse
## 1 0.05974883 0.1381892 0.01909625
## 2 0.07214633 0.1388670 0.01928404
## 3 0.10418698 0.1594435 0.02542223
## 4 0.08216515 0.1521288 0.02314317
## 5 0.12105102 0.1727169 0.02983113
## 6 0.11516887 0.1695383 0.02874324
```

H2O AutoML on the iris data

```
iris_automl@leader
```

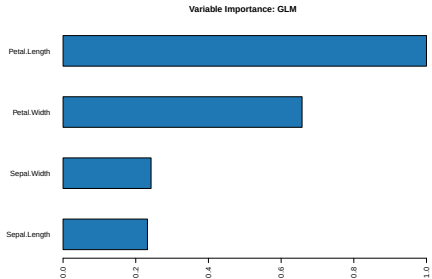
```
## Model Details:
## ==============
##
## H2OMultinomialModel: glm
## Model ID:  GLM_1_AutoML_3_20261014_95159
## GLM Model: summary
##        family        link              regularization
## 1 multinomial multinomial Ridge ( lambda = 1.067E-4 )
##                                                                    lambda_search
## 1 nlambda = 30, lambda.max = 41.141, lambda.min = 1.067E-4, lambda.1se = 7.172E-4
##   number_of_predictors_total number_of_active_predictors number_of_iterations
## 1                         15                          12                  125
##                                training_frame
## 1 AutoML_3_20261014_95159_training_RTMP_sid_8911_4
##
## Coefficients: glm multinomial coefficients
##          names coefs_class_0 coefs_class_1 coefs_class_2 std_coefs_class_0
## 1    Intercept     11.828863     -4.708316    -30.814973         -6.429657
## 2 Sepal.Length     -2.524279      3.057256     -0.277636         -2.007362
## 3  Sepal.Width      5.960968     -0.502229     -5.166601          2.490298
## 4 Petal.Length     -3.343034     -2.273906      6.203857         -5.674897
## 5  Petal.Width     -6.358918     -1.681789      9.595906         -4.757884
##   std_coefs_class_1 std_coefs_class_2
## 1          0.674038        -11.228257
## 2          2.431197         -0.220782
## 3         -0.209815         -2.158438
```

H2O AutoML on the iris data

```
h2o.varimp(iris_automl@leader)
```

```
## Variable Importances:
##       variable relative_importance scaled_importance percentage
## 1 Petal.Length           20.066147          1.000000   0.469053
## 2  Petal.Width           13.196106          0.657630   0.308463
## 3  Sepal.Width            4.858551          0.242127   0.113570
## 4 Sepal.Length            4.659341          0.232199   0.108914
```

```
h2o.varimp_plot(iris_automl@leader)
```



H2O AutoML on the iris data

```
pred <- h2o.predict(iris_automl@leader, valid)
```

```
##   |                                                                              |
```

```
pred
```

```
##   predict    setosa   versicolor    virginica
## 1  setosa 0.9997415 0.0002585061 5.397577e-24
## 2  setosa 0.9997658 0.0002342450 3.908678e-22
## 3  setosa 0.9992174 0.0007825571 1.554512e-22
## 4  setosa 0.9995899 0.0004101071 1.477655e-26
## 5  setosa 0.9995874 0.0004125999 2.660977e-23
## 6  setosa 0.9976755 0.0023245095 6.363392e-23
##
## [40 rows x 4 columns]
```

H2O AutoML on the prostate cancer dataset

```
prostate_path <- system.file("extdata", "prostate.csv",
                             package = "h2o")
prostate <- h2o.importFile(path = prostate_path, header = TRUE)
```

```
##   |                                                                      |
```

```
y <- "CAPSULE"
prostate[,y] <- as.factor(prostate[,y])
knitr::kable(prostate)
```

ID	CAPSULE	AGE	RACE	DPROS	DCAPS	PSA	VOL	GLEASON
1	0	65	1	2	1	1.40	0.00	6
2	0	72	1	3	2	6.70	0.00	7
3	0	70	1	1	2	4.90	0.00	6
4	0	76	2	2	1	51.20	20.00	7
5	0	69	1	1	1	12.30	55.90	6
6	1	71	1	3	2	3.30	0.00	8
7	0	68	2	4	2	31.90	0.00	7
8	0	61	2	4	2	66.70	27.20	7
9	0	69	1	1	1	3.90	24.00	7
10	0	68	2	1	2	13.00	0.00	6
11	1	68	2	4	2	4.00	0.00	7
12	1	72	1	2	2	21.20	0.00	7
13	1	72	1	4	2	22.70	0.00	9
14	1	65	1	4	2	39.00	0.00	7
15	0	75	1	1	1	7.50	0.00	5
16	0	73	1	2	1	2.60	0.00	5

H2O AutoML on the prostate cancer dataset

```
aml <- h2o.automl(y = y, training_frame = prostate,
                  max_runtime_secs = 20, seed = 1)
```

```
##   |                                                                              |
## 09:52:23.610: AutoML: XGBoost is not available; skipping it.  |
```

```
lb <- h2o.get_leaderboard(aml)
head(lb)
```

```
##                                              model_id       auc   logloss
## 1           GBM_grid_1_AutoML_4_20261014_95223_model_11 0.8144309 0.5289923
## 2 StackedEnsemble_BestOfFamily_4_AutoML_4_20261014_95223 0.8142006 0.5192106
## 3    StackedEnsemble_AllModels_2_AutoML_4_20261014_95223 0.8135095 0.5173286
## 4            GBM_grid_1_AutoML_4_20261014_95223_model_4 0.8123290 0.5222005
## 5 StackedEnsemble_BestOfFamily_6_AutoML_4_20261014_95223 0.8102560 0.5221448
## 6    StackedEnsemble_AllModels_1_AutoML_4_20261014_95223 0.8091331 0.5216481
##       aucpr mean_per_class_error      rmse       mse
## 1 0.7171983            0.2298379 0.4190206 0.1755782
## 2 0.7303591            0.2368777 0.4153629 0.1725263
## 3 0.7348310            0.2424203 0.4152259 0.1724125
## 4 0.7152434            0.2413550 0.4169814 0.1738735
## 5 0.7357463            0.2434137 0.4163154 0.1733185
## 6 0.7373510            0.2436296 0.4172881 0.1741293
```

Summary

- H2O: open-source, in-memory, distributed ML with an R interface
- AutoML trains and tunes many models automatically
- The leaderboard ranks models by cross-validated performance
- AutoML speeds up model search, but you still own the question, data, and validation

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Denver
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] h2o_3.44.0.3    lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
##  [5] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
##  [9] tibble_3.3.1    tidyverse_2.0.0 caret_7.0-1     lattice_0.22-9
## [13] ggplot2_4.0.3
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1     timeDate_4052.112    farver_2.1.2
##  [4] S7_0.2.2             bitops_1.0-9         fastmap_1.2.0
```